# IssueManagementModal UI設計書

## メタデータ

- **コンポーネントID**: IssueManagementModal
- **コンポーネント名**: 問題管理モーダル
- **ファイルパス**: `frontend/src/components/ExperimentWorker/IssueManagementModal.tsx`
- **利用権限**: 実験作業者（experiment_worker）
- **作成日**: 2026-01-19
- **最終更新日**: 2026-03-11

---

## 1. 概要

### 1.1 コンポーネントの目的

トランジションに関連する問題の一覧表示、詳細閲覧、新規問題報告、コメント追加、ステータス変更機能を統合的に提供するモーダルです。問題の全体像を把握し、効率的に管理できます。

### 1.2 主要機能

- 問題一覧表示（検索・フィルタリング）
- 問題詳細表示（タイトル、内容、ステータス、コメント履歴）
- 新規問題報告フォーム
- コメント追加
- ステータス変更（未解決/解決済）
- 参照専用モード対応
- 承認依頼中の編集制限

### 1.3 使用場所

- ProgressCheckTab（進捗確認タブ）
- InstructionConfirmationTab（指示確認タブ）
- その他、トランジション関連の問題管理が必要な場所

---

# 2. コンポーネント構成

## 2.1 画面イメージ







## 2.2 レイアウト構造

```
┌─Modal（幅:1200px）──────────────────────────────────────┐
│ Header: [ExclamationCircleOutlined] 問題報告 - {トランジション名} │
├──────────────────────────────────────────────────────┤
│ [新規問題報告] ボタン（viewMode時は非表示）                  │
├──────────────────────────────────────────────────────┤
│ Body: 2カラムレイアウト（Row/Col）                        │
│ ┌─左側（Col span=10）─────┐ ┌─右側（Col span=14）─────┐ │
│ │ Card: 問題一覧            │ │ Card: 問題詳細 or 新規  │ │
│ │ ┌─検索・フィルター──┐     │ │                        │ │
│ │ │ [検索...]        │     │ │ ┌─問題詳細──────┐   │ │
│ │ │ [ステータス▼]    │     │ │ │ タイトル       │   │ │
│ │ └──────────────┘     │ │ │ 状態・対象     │   │ │
│ │                          │ │ │ 報告者・時刻   │   │ │
│ │ ┌─Table──────┐       │ │ │ 内容           │   │ │
│ │ │ ○ 問題A       │       │ │ │                │   │ │
│ │ │   [未解決]    │       │ │ │ コメント一覧   │   │ │
│ │ │ ◉ 問題B (選択中) │       │ │ │                │   │ │
│ │ │   [解決済]    │       │ │ │ [コメント入力] │   │ │
│ │ │ ○ 問題C       │       │ │ │                │   │ │
│ │ │   [未解決]    │       │ │ │ [送信] [未解決] │   │ │
│ │ │ ...           │       │ │ │ [解決]         │   │ │
│ │ └──────────────┘       │ │ └──────────────┘   │ │
│ └──────────────────────┘ └──────────────────────┘ │
│ 高さ: calc(70vh - 100px)                              │
└──────────────────────────────────────────────────────┘

モーダル幅: 1200px
```

```
モーダルトップ位置: 20px
destroyOnHidden: true
```

## 2.3 使用コンポーネント

### Ant Design コンポーネント

- `Modal` - モーダルダイアログ
- `Row` / `Col` - 2カラムレイアウト
- `Card` - 問題一覧カード、問題詳細カード
- `Table` - 問題一覧テーブル
- `List` - コメント一覧
- `Form` - 新規問題報告フォーム
- `Form.Item` - フォーム項目
- `Input` - 検索、タイトル入力
- `Input.TextArea` - 内容入力、コメント入力
- `Select` - ステータスフィルター、対象選択、問題タイプ選択
- `Button` - 各種ボタン
- `Tag` - ステータスタグ
- `Space` - 要素間隔制御
- `Typography.Text` / `Typography.Title` - テキスト表示
- `Divider` - 区切り線
- `message` - トースト通知
- `Spin` - 問題一覧ローディング表示

### Reactフック

- `useState` - 各種状態管理
- `useMemo` - 派生値の計算（filteredIssues, selectedIssue）
- `useEffect` - モーダルオープン時の問題一覧取得

- `useCallback` - fetchIssues関数のメモ化

### アイコン

- `ExclamationCircleOutlined` - 未解決アイコン
- `CheckCircleOutlined` - 解決済アイコン
- `PlusOutlined` - 新規問題報告アイコン

---

## 3. Props

```
interface IssueManagementModalProps {
  visible: boolean;                    // モーダルの表示状態
  /** タイトルに表示する名前 */
  targetName: string;
  /** 問題一覧取得・新規報告に必要なプロジェクトID */
  projectId: string;
  /** コメント追加・ステータス変更に必要なプロジェクト名 */
  projectName: string;
  /** オプション: 省略時はプロジェクト全件を取得 */
  methodId?: string;
  /** オプション: タスク単位の絞り込みに使用 */
  taskNo?: number;
  /** オプション: トランジション単位の絞り込みに使用 */
  transitionId?: string;
  onClose: () => void;                 // 閉じるイベントハンドラ
  viewMode?: boolean;                  // 参照専用モード（デフォルト: false）
  filterTaskId?: string;               // フィルタリング用タスクID
  projectStatus?: ProjectStatus | null; // プロジェクトステータス（編集可否の判定用）
}

interface Issue {
  issueId: string;
  title: string;
  content: string;
  status: IssueStatus;
  targetType: IssueTargetType;
  projectName?: string;
  taskName?: string;
  transitionName?: string;
  reportedBy?: { userName: string };
  reportedAt: string;
```

```
  comments: IssueComment[];
  taskId?: string;
}

type IssueStatus = 'unresolved' | 'resolved';
type IssueTargetType = 'project' | 'task' | 'transition';
type ProjectStatus = 'draft' | 'waiting_approval' | 'active' | 'suspended' | 'completed';

interface IssueComment {
  content: string;
  createdBy: { userName: string };
  createdAt: string;
}

interface IssueReportFormData {
  title: string;
  targetType: IssueTargetType;
  issueType: string;
  content: string;
}
```

**Props詳細:**

- `visible` : モーダルの表示/非表示を制御
- `targetName` : タイトルに表示する名前（トランジション名など）
- `projectId` : 問題一覧取得・新規報告に使用するプロジェクトID
- `projectName` : コメント追加・ステータス変更APIに使用するプロジェクト名
- `methodId` : 絞り込み用メソッドID（オプション、省略時はプロジェクト全件を取得）
- `taskNo` : タスク単位の絞り込みに使用するタスク番号（オプション）
- `transitionId` : トランジション単位の絞り込みに使用するID（オプション）
- `onClose` : モーダルを閉じるハンドラ
- `viewMode` : true の場合、参照専用モード（編集・新規作成不可）
- `filterTaskId` : 指定した場合、そのタスクIDの問題のみ表示
- `projectStatus` : プロジェクトのステータス（'waiting_approval'の場合は編集不可）

> ***設計変更について:*** *旧設計では* `issues` *Props として親コンポーネントから問題一覧を受け取り、* `onSubmitIssue` */* `onSubmitComment` */* `onUpdateStatus` *コールバックを通じて操作していた。 現行設計では、コンポーネント内部で* `issueApiService` *を直接呼び出す設計に変更されており、親コンポーネントからのデータ受け渡しは不要になっている。*

# 4. State（内部状態）

## 4.1 状態変数

```
const [form] = Form.useForm();                                    // 新規問題報告フォーム
const [commentForm] = Form.useForm();                             // コメントフォーム

// 問題一覧（内部state: コンポーネント内部でAPIから取得）
const [issues, setIssues] = useState<Issue[]>([]);
const [issuesLoading, setIssuesLoading] = useState(false);

// 選択中の問題IDのみをstateで保持する
// selectedIssue 自体は issues state から useMemo で導出することで、
// 再取得のたびに自動的に最新データが反映される
const [selectedIssueId, setSelectedIssueId] = useState<string | null>(null);  // 選択中の問題ID

const [showNewIssueForm, setShowNewIssueForm] = useState(false);          // 新規報告フォーム表示
状態
const [newComment, setNewComment] = useState('');                         // 新規コメントテキスト

// フィルタリング用
const [searchText, setSearchText] = useState('');                         // 検索テキスト
const [statusFilter, setStatusFilter] = useState<IssueStatus | 'all'>('all');  // ステータスフ
ィルター

// 問題一覧を再取得する関数
const fetchIssues = useCallback(async () => {
  if (!projectId) return;
  setIssuesLoading(true);
  try {
    const data = await getIssues({ projectId, methodId, taskNo, transitionId });
    setIssues(data);
  } catch (error) {
    console.error('[IssueManagementModal] fetchIssues error:', error);
    message.error('問題一覧の取得に失敗しました');
  } finally {
    setIssuesLoading(false);
  }
}, [projectId, methodId, taskNo, transitionId]);

// モーダルが開いた時に問題一覧を取得する
useEffect(() => {
  if (visible) {
    fetchIssues();
```

```
  }
}, [visible, fetchIssues]);

// 選択中の問題を issues state から導出する（useMemo）
// issues が更新（コメント送信・ステータス変更後の再取得）されると
// selectedIssue も自動的に最新データに追随し、コメント一覧が即時更新される
const selectedIssue = useMemo(
  () => issues.find((issue) => issue.issueId === selectedIssueId) ?? null,
  [issues, selectedIssueId]
);
```

**設計ポイント:**

- `issues` はPropsではなく内部Stateとして保持し、コンポーネント内部でAPIから取得する
- `selectedIssue` はstateではなく `useMemo` で導出される派生値
- `issues` stateが変化するたびに自動再計算されるため、コメント送信後の即時表示が実現される
- `fetchIssues` は `useCallback` でメモ化し、`useEffect` の依存配列に安全に渡す
- 各操作（問題報告・コメント送信・ステータス変更）後に `fetchIssues()` を再呼び出しして最新状態を反映する

## 4.2 使用APIサービス（issueApiService）

このコンポーネントは `@/services/issueApiService` を直接インポートして使用します。

```
import {
  getIssues,
  createIssue,
  addComment,
  updateIssueStatus,
} from '@/services/issueApiService';
```

| 関数 | 用途 | 呼び出しタイミング |
| --- | --- | --- |
| `getIssues({ projectId, methodId, taskNo, transitionId })` | 問題一覧取得 | モーダルオープン時・各操作後 |
| `createIssue({ projectId, methodId, issueType, targetType, taskNo, transitionId, title, description })` | 新規問題報告 | 保存ボタンクリック時 |

| | | |
|---|---|---|
| `addComment({ projectName, issueId, comment })` | コメント追加 | コメント送信・ステータス変更時 |
| `updateIssueStatus({ projectName, issueId, status })` | ステータス変更 | 未解決/解決ボタンクリック時 |

**再取得パターン:**

各操作完了後は必ず `fetchIssues()` を呼び出し、最新状態を反映します。

```
ユーザー操作
  ↓
APIを直接呼び出し（createIssue / addComment / updateIssueStatus）
  ↓
成功メッセージ表示
  ↓
fetchIssues() で一覧を再取得
  ↓
issues state が更新
  ↓
selectedIssue（useMemo）が自動的に最新状態に追随
```

## 4.3 編集可否の判定

```
// プロジェクトが承認依頼中の場合は編集不可
const isApprovalPending = projectStatus === 'waiting_approval';
const isEditDisabled = viewMode || isApprovalPending;
```

**編集不可条件:**

- `viewMode === true`（参照専用モード）
- `projectStatus === 'waiting_approval'`（承認依頼中）

---

# 5. データフィルタリング

## 5.1 フィルタリングロジック

```
const filteredIssues = useMemo(() => {
  return issues.filter((issue) => {
    // タスクIDフィルタ（参照専用モード用）
    if (filterTaskId && issue.taskId !== filterTaskId) {
      return false;
    }

    // あいまい検索（タイトル、内容）
    const matchesSearch =
      searchText === '' ||
      issue.title.toLowerCase().includes(searchText.toLowerCase()) ||
      issue.content.toLowerCase().includes(searchText.toLowerCase());

    // ステータスフィルタ
    const matchesStatus =
      statusFilter === 'all' || issue.status === statusFilter;

    return matchesSearch && matchesStatus;
  });
}, [issues, searchText, statusFilter, filterTaskId]);
```

**フィルター条件:**

1. **タスクIDフィルター**（`filterTaskId`が指定されている場合）
   - 指定されたタスクIDの問題のみ表示
2. **あいまい検索**（`searchText`）
   - タイトル（部分一致、大文字小文字無視）
   - 内容（部分一致、大文字小文字無視）
3. **ステータスフィルター**（`statusFilter`）
   - `'all'`: すべて表示
   - `'unresolved'`: 未解決のみ
   - `'resolved'`: 解決済のみ

**論理演算:**

- 全ての条件をAND結合

# 6. 問題一覧（左側カラム）

## 6.1 検索・フィルターUI

```
<Space direction="vertical" size="small" style={{ width: '100%', marginBottom: 12 }}>
  <Input
    placeholder="タイトル、内容で検索..."
    value={searchText}
    onChange={(e) => setSearchText(e.target.value)}
    allowClear
    style={{ width: '100%' }}
  />
  <Select
    placeholder="ステータスで絞り込み"
    value={statusFilter}
    onChange={(value) => setStatusFilter(value)}
    style={{ width: '100%' }}
    allowClear
    onClear={() => setStatusFilter('all')}
  >
    <Select.Option value="all">すべて</Select.Option>
    <Select.Option value="unresolved">未解決</Select.Option>
    <Select.Option value="resolved">解決済</Select.Option>
  </Select>
</Space>
```

## 6.2 テーブル列定義（2行表示）

```
const columns: ColumnsType<Issue> = [
  {
    title: '問題情報',
    key: 'issueInfo',
    render: (_, record: Issue) => (
      <div style={{ padding: '4px 0' }}>
        {/* 上段: タイトル */}
        <div style={{ marginBottom: 8 }}>
          <Text strong style={{ fontSize: '14px' }}>
            {record.title}
          </Text>
        </div>
```

```
        {/* 下段: 状態と対象 */}
        <Space size="middle">
          {getStatusTag(record.status)}
          <Text type="secondary" style={{ fontSize: '12px' }}>
            {getTargetTypeText(record.targetType)}
            {getTargetName(record) && (
              <>: {getTargetName(record)}</>
            )}
          </Text>
        </Space>
      </div>
    ),
  },
];
```

**表示内容:**

- **上段**: タイトル（太字、14px）
- **下段**: ステータスタグ + 対象タイプと対象名（グレー、12px）

## 6.3 行クリックイベント

```
onRow={(record) => ({
  onClick: () => handleSelectIssue(record),
  style: {
    cursor: 'pointer',
    backgroundColor:
      selectedIssue?.issueId === record.issueId
        ? '#e6f7ff'
        : undefined,
  },
})}
```

**動作:**

- 行クリックで問題を選択
- 選択行は背景色 `#e6f7ff`（水色）
- カーソルはポインター

## 6.4 ステータスタグの表示

```
const getStatusTag = (status: IssueStatus) => {
  if (status === 'resolved') {
    return (
      <Tag icon={<CheckCircleOutlined />} color="success">
        解決済
      </Tag>
    );
  }
  return (
    <Tag icon={<ExclamationCircleOutlined />} color="warning">
      未解決
    </Tag>
  );
};
```

| ステータス | アイコン | カラー | 表示テキスト |
|---|---|---|---|
| resolved | CheckCircleOutlined | success（緑） | 解決済 |
| unresolved | ExclamationCircleOutlined | warning（オレンジ） | 未解決 |

---

# 7. 問題詳細（右側カラム）

## 7.1 表示状態の切り替え

右側カラムには以下の3つの状態があります：

1. **問題未選択時**: 「問題を選択してください」テキスト
2. **新規問題報告時**: 新規問題報告フォーム
3. **問題選択時**: 問題詳細表示

## 7.2 問題詳細の表示内容

### 基本情報

```
<Space direction="vertical" size="small" style={{ width: '100%', marginBottom: 16 }}>
  {/* タイトル */}
```

```
  <div>
    <Text type="secondary">タイトル:</Text>
    <Title level={5}>{selectedIssue.title}</Title>
  </div>

  {/* 状態と対象 */}
  <Space size="large">
    <div>
      <Text type="secondary">状態: </Text>
      {getStatusTag(selectedIssue.status)}
    </div>
    <div>
      <Text type="secondary">対象: </Text>
      <Text>{getTargetTypeText(selectedIssue.targetType)}: {getTargetName(selectedIssue)}
</Text>
    </div>
  </Space>

  {/* 報告者と時刻 */}
  <Space size="large">
    <div>
      <Text type="secondary">報告者: </Text>
      <Text>{selectedIssue.reportedBy?.userName || '-'}</Text>
    </div>
    <div>
      <Text type="secondary">報告時刻: </Text>
      <Text>{new Date(selectedIssue.reportedAt).toLocaleString('ja-JP')}</Text>
    </div>
  </Space>
</Space>
```

## 内容

```
<div style={{ marginBottom: 16 }}>
  <Text type="secondary" strong>内容:</Text>
  <div style={{
    marginTop: 8,
    padding: 12,
    backgroundColor: '#fafafa',
    borderRadius: 4,
    whiteSpace: 'pre-wrap',
  }}>
    {selectedIssue.content}
  </div>
</div>
```

## コメント一覧

```
<div style={{ marginBottom: 16 }}>
  <Text type="secondary" strong>
    コメント: ({selectedIssue.comments.length})
  </Text>
  <List
    style={{ marginTop: 8, maxHeight: '200px', overflow: 'auto' }}
    dataSource={selectedIssue.comments}
    locale={{ emptyText: 'コメントがありません' }}
    renderItem={(comment: IssueComment) => (
      <List.Item style={{ padding: '8px 0' }}>
        <div style={{ width: '100%' }}>
          {/* ユーザー名と時刻 */}
          <div style={{ marginBottom: 4 }}>
            <Text strong style={{ fontSize: '13px' }}>
              {comment.createdBy.userName}
            </Text>
            <Text type="secondary" style={{ fontSize: '12px', marginLeft: 8 }}>
              {new Date(comment.createdAt).toLocaleString('ja-JP')}
            </Text>
          </div>
          {/* コメント内容 */}
          <div style={{
            padding: 8,
            backgroundColor: '#f5f5f5',
            borderRadius: 4,
            whiteSpace: 'pre-wrap',
          }}>
            {comment.content}
          </div>
        </div>
      </List.Item>
    )}
  />
</div>
```

**特徴:**

- コメント数を表示
- 最大高さ200px、スクロール可能
- 各コメントは投稿者名、投稿時刻、内容を表示
- 投稿時刻は日本語ロケールでフォーマット

## コメント入力（viewModeでは非表示）

```
<div style={{ marginBottom: 16 }}>
  <Text type="secondary" strong>コメントを追加</Text>
  <TextArea
    value={newComment}
    onChange={(e) => setNewComment(e.target.value)}
    rows={3}
    placeholder={isApprovalPending ? 'プロジェクトが承認依頼中のため、コメントを追加できません' :
'コメントを入力...'}
    maxLength={500}
    showCount
    style={{ marginTop: 8 }}
    disabled={isEditDisabled}
  />
</div>
```

**設定:**

- 行数: 3行
- 最大文字数: 500文字
- 文字数カウント表示: あり
- 承認依頼中は非活性

## ボタンエリア（viewModeでは非表示）

```
<div style={{ display: 'flex', justifyContent: 'flex-end', gap: '8px' }}>
  <Button
    type="primary"
    onClick={handleCommentSubmit}
    disabled={!newComment.trim() || isEditDisabled}
  >
    コメント送信
  </Button>
  <Button
    onClick={handleSetUnresolved}
    disabled={!newComment.trim() || selectedIssue.status === 'unresolved' || isEditDisabled}
  >
    未解決
  </Button>
  <Button
```

```
    onClick={handleSetResolved}
    disabled={!newComment.trim() || selectedIssue.status === 'resolved' || isEditDisabled}
    style={{
      backgroundColor: newComment.trim() && selectedIssue.status === 'unresolved' &&
!isEditDisabled ? '#52c41a' : undefined,
      borderColor: newComment.trim() && selectedIssue.status === 'unresolved' &&
!isEditDisabled ? '#52c41a' : undefined,
      color: newComment.trim() && selectedIssue.status === 'unresolved' && !isEditDisabled ?
'#fff' : undefined,
    }}
  >
    解決
  </Button>
</div>
```

**ボタンの活性条件:**

| ボタン | 活性条件 |
|---|---|
| コメント送信 | コメントが入力されている かつ 編集可能 |
| 未解決 | コメントが入力されている かつ 現在のステータスが「解決済」 かつ 編集可能 |
| 解決 | コメントが入力されている かつ 現在のステータスが「未解決」 かつ 編集可能 |

**ボタンのtooltip（承認依頼中）:**

承認依頼中（`isApprovalPending === true`）の場合、各ボタンに以下の `title` tooltipが表示されます：

| ボタン | tooltip |
|---|---|
| コメント送信 | プロジェクトが承認依頼中のため、コメント送信はできません |
| 未解決 | プロジェクトが承認依頼中のため、ステータス変更はできません |
| 解決 | プロジェクトが承認依頼中のため、ステータス変更はできません |

---

# 8. 新規問題報告フォーム

## 8.1 フォーム項目

```
<Form form={form} layout="vertical">
  <Form.Item
    label="タイトル"
    name="title"
    rules={[
      { required: true, message: 'タイトルを入力してください' },
      { max: 100, message: 'タイトルは100文字以内で入力してください' },
    ]}
  >
    <Input placeholder="問題のタイトルを入力" maxLength={100} />
  </Form.Item>

  <Form.Item
    label="対象"
    name="targetType"
    initialValue="transition"
    rules={[{ required: true, message: '対象を選択してください' }]}
  >
    <Select>
      <Select.Option value="project">プロジェクト</Select.Option>
      <Select.Option value="task">タスク</Select.Option>
      <Select.Option value="transition">トランジション</Select.Option>
    </Select>
  </Form.Item>

  <Form.Item
    label="問題タイプ"
    name="issueType"
    rules={[{ required: true, message: '問題タイプを選択してください' }]}
  >
    <Select
      placeholder="問題タイプを選択"
      loading={issueTypeLoading}
      options={issueTypeOptions}
    />
  </Form.Item>

  <Form.Item
    label="内容"
    name="content"
    rules={[
      { required: true, message: '内容を入力してください' },
      { min: 10, message: '内容は10文字以上で入力してください' },
    ]}
  >
    <TextArea
      rows={8}
      placeholder="問題の詳細を具体的に入力してください"
      maxLength={1000}
```

```
        showCount
      />
    </Form.Item>

    <Form.Item style={{ marginBottom: 0 }}>
      <Space style={{ float: 'right' }}>
        <Button onClick={() => setShowNewIssueForm(false)}>
          キャンセル
        </Button>
        <Button type="primary" onClick={handleSubmitNewIssue}>
          保存
        </Button>
      </Space>
    </Form.Item>
  </Form>
```

## 8.2 フォーム項目詳細

| 項目 | 入力タイプ | バリデーション | 初期値 |
|---|---|---|---|
| タイトル | Input | 必須、最大100文字 | - |
| 対象 | Select | 必須 | 'transition' |
| 問題タイプ | Select | 必須 | - |
| 内容 | TextArea | 必須、最小10文字、最大1000文字 | - |

## 8.3 問題タイプの選択肢

問題タイプの選択肢は **コードマスターAPI** から動的に取得します。ハードコードはしていません。

**API:** `GET /api/v1/code-master/issue_type/options`

**取得方法:** `useCodeMaster('issue_type')` カスタムフックを使用

```
// コンポーネント内で自動取得
const { options: issueTypeOptions, loading: issueTypeLoading } = useCodeMaster('issue_type');
```

**選択肢の管理:** `code_master` テーブルの `code_division = 'issue_type'` レコードで管理します。 選択肢の追加・変更・削除はDBのみで対応可能です（フロントエンドの変更不要）。

---

# 9. イベント

## 9.1 イベント一覧

| イベント名 | トリガー | 処理内容 |
|---|---|---|
| handleSelectIssue | 問題行クリック | 問題を選択して詳細を表示 |
| handleNewIssueClick | 新規問題報告ボタンクリック | 新規報告フォームを表示 |
| handleSubmitNewIssue | 保存ボタンクリック | 新規問題を報告 |
| handleCommentSubmit | コメント送信ボタンクリック | コメントを送信 |
| handleSetUnresolved | 未解決ボタンクリック | ステータスを「未解決」に変更してコメントを送信 |
| handleSetResolved | 解決ボタンクリック | ステータスを「解決済」に変更してコメントを送信 |
| handleClose | キャンセルボタンクリック | モーダルを閉じて状態をリセット |

## 9.2 イベント詳細

### handleSubmitNewIssue（新規問題報告）

```
const handleSubmitNewIssue = async () => {
  try {
    const values = await form.validateFields();

    await createIssue({
      projectId,
      methodId,
      issueType: values.issueType,
      targetType: values.targetType,
      taskNo,
      transitionId,
      title: values.title,
      description: values.content,
    });

    message.success('問題を報告しました');
    form.resetFields();
    setShowNewIssueForm(false);
    // 一覧を再取得
    await fetchIssues();
```

```
  } catch (error) {
    if (error && typeof error === 'object' && 'errorFields' in error) {
      // Form validation error - do nothing
      return;
    }
    console.error('[IssueManagementModal] handleSubmitNewIssue error:', error);
    message.error('問題の報告に失敗しました');
  }
};
```

**処理フロー:**

1. フォームのバリデーション
2. `createIssue()` APIを直接呼び出し
3. 成功メッセージ表示
4. フォームをリセット
5. 新規報告フォームを閉じる
6. `fetchIssues()` で一覧を再取得

## handleCommentSubmit（コメント送信）

```
const handleCommentSubmit = async () => {
  if (!selectedIssue) return;

  if (!newComment.trim()) {
    message.warning('コメントを入力してください');
    return;
  }

  try {
    await addComment({
      projectName,
      issueId: selectedIssue.issueId,
      comment: newComment.trim(),
    });

    message.success('コメントを送信しました');
    setNewComment('');
    commentForm.resetFields();
    // 一覧を再取得（コメントが追加された状態を反映）
    await fetchIssues();
  } catch (error) {
    console.error('[IssueManagementModal] handleCommentSubmit error:', error);
```

```
    message.error('コメントの送信に失敗しました');
  }
};
```

**処理フロー:**

1. 問題選択確認
2. コメント入力確認
3. `addComment()` APIを直接呼び出し
4. 成功メッセージ表示
5. コメントをクリア
6. `fetchIssues()` で一覧を再取得

## handleSetUnresolved（未解決に変更）

```
const handleSetUnresolved = async () => {
  if (!selectedIssue) return;

  if (!newComment.trim()) {
    message.warning('コメントを入力してください');
    return;
  }

  try {
    // コメントを送信してからステータスを更新
    await addComment({
      projectName,
      issueId: selectedIssue.issueId,
      comment: newComment.trim(),
    });
    await updateIssueStatus({
      projectName,
      issueId: selectedIssue.issueId,
      status: 'unresolved',
    });

    message.success('ステータスを「未解決」に変更しました');
    setNewComment('');
    commentForm.resetFields();
    // 一覧を再取得
    await fetchIssues();
  } catch (error) {
    console.error('[IssueManagementModal] handleSetUnresolved error:', error);
```

```
    message.error('ステータスの変更に失敗しました');
  }
};
```

**処理フロー:**

1. 問題選択確認
2. コメント入力確認
3. `addComment()` APIを直接呼び出し
4. `updateIssueStatus()` APIを直接呼び出し（`'unresolved'` を渡す）
5. 成功メッセージ表示
6. コメントをクリア
7. `fetchIssues()` で一覧を再取得

## handleSetResolved（解決済に変更）

```
const handleSetResolved = async () => {
  if (!selectedIssue) return;

  if (!newComment.trim()) {
    message.warning('コメントを入力してください');
    return;
  }

  try {
    // コメントを送信してからステータスを更新
    await addComment({
      projectName,
      issueId: selectedIssue.issueId,
      comment: newComment.trim(),
    });
    await updateIssueStatus({
      projectName,
      issueId: selectedIssue.issueId,
      status: 'resolved',
    });

    message.success('ステータスを「解決済」に変更しました');
    setNewComment('');
    commentForm.resetFields();
    // 一覧を再取得
    await fetchIssues();
```

```
  } catch (error) {
    console.error('[IssueManagementModal] handleSetResolved error:', error);
    message.error('ステータスの変更に失敗しました');
  }
};
```

**処理フロー:**

1. 問題選択確認
2. コメント入力確認
3. `addComment()` APIを直接呼び出し
4. `updateIssueStatus()` APIを直接呼び出し（`'resolved'` を渡す）
5. 成功メッセージ表示
6. コメントをクリア
7. `fetchIssues()` で一覧を再取得

## handleClose（モーダルを閉じる）

```
const handleClose = () => {
  setSelectedIssueId(null);
  setShowNewIssueForm(false);
  form.resetFields();
  commentForm.resetFields();
  setNewComment('');
  setIssues([]);
  onClose();
};
```

**処理フロー:**

1. 選択中の問題IDをクリア
2. 新規報告フォームを非表示
3. フォームをリセット
4. コメントフォームをリセット
5. コメントをクリア
6. 問題一覧をクリア

7. `onClose` コールバック実行

---

# 10. ヘルパー関数

## 10.1 対象タイプの表示テキスト取得

```
const getTargetTypeText = (targetType: IssueTargetType) => {
  const typeMap = {
    project: 'プロジェクト',
    task: 'タスク',
    transition: 'トランジション',
  };
  return typeMap[targetType] || targetType;
};
```

## 10.2 対象名の取得

```
const getTargetName = (issue: Issue): string => {
  switch (issue.targetType) {
    case 'project':
      return issue.projectName || '';
    case 'task':
      return issue.taskName || '';
    case 'transition':
      return issue.transitionName || '';
    default:
      return '';
  }
};
```

---

# 11. スタイリング

## 11.1 モーダルスタイル

```
<Modal
  title={
    <Space>
      <ExclamationCircleOutlined style={{ color: '#faad14' }} />
      <span>
        {viewMode ? '問題確認' : '問題報告'} - {targetName}
      </span>
    </Space>
  }
  open={visible}
  onCancel={handleClose}
  footer={null}
  width={1200}
  style={{ top: 20 }}
  destroyOnHidden
>
```

**特徴:**

- モーダル幅: 1200px
- トップ位置: 20px（画面上部に近づける）
- カスタムタイトル: 警告アイコン（オレンジ色）+ テキスト
  - `viewMode` が `true` の場合: `問題確認 - {targetName}`
  - `viewMode` が `false` の場合: `問題報告 - {targetName}`
- フッターなし（`footer={null}`）
- `destroyOnHidden={true}`: モーダルが非表示になる度にコンポーネントを破棄（`destroyOnClose` の後継API）

## 11.2 カードスタイル

```
// 左側カード（問題一覧）
<Card
  title="問題一覧"
  size="small"
  style={{ height: 'calc(70vh - 100px)' }}
  styles={{ body: { padding: 12, overflow: 'auto', height: 'calc(100% - 38px)' } }}
>
```

```
// 右側カード（問題詳細）
<Card
  title={showNewIssueForm ? '新規問題報告' : '問題詳細'}
  size="small"
  style={{ height: 'calc(70vh - 100px)' }}
  styles={{ body: { padding: 16, overflow: 'auto', height: 'calc(100% - 38px)' } }}
>
```

**カードの高さ:**

- 全体: `calc(70vh - 100px)`（ビューポート高さの70%から100pxを引く）
- ボディ: `calc(100% - 38px)`（カードヘッダー分を除く）
- オーバーフロー: auto（スクロール可能）

## 11.3 選択行のハイライト

```
style: {
  cursor: 'pointer',
  backgroundColor:
    selectedIssue?.issueId === record.issueId
      ? '#e6f7ff'
      : undefined,
},
```

**選択行:**

- 背景色: `#e6f7ff`（水色）
- カーソル: pointer

## 11.4 解決ボタンのカスタムスタイル

```
style={{
  backgroundColor: newComment.trim() && selectedIssue.status === 'unresolved' &&
!isEditDisabled ? '#52c41a' : undefined,
  borderColor: newComment.trim() && selectedIssue.status === 'unresolved' && !isEditDisabled ?
'#52c41a' : undefined,
  color: newComment.trim() && selectedIssue.status === 'unresolved' && !isEditDisabled ?
```

```
'#fff' : undefined,
}}
```

**動的スタイル:**

- 活性時（コメント入力済 かつ 未解決 かつ 編集可能）: 緑色の背景とボーダー、白文字
- 非活性時: デフォルトスタイル

---

# 12. アクセシビリティ

## 12.1 キーボード操作

- Tabキーでフォーカス移動
- Enterキーでボタン実行、問題選択
- Escapeキーでモーダルを閉じる

## 12.2 フォーカス管理

- モーダル表示時、最初のフォーム要素に自動フォーカス
- 問題一覧の行はキーボードで選択可能
- ボタンの活性/非活性状態が視覚的に明確

## 12.3 承認依頼中のヘルプテキスト

```
title={isApprovalPending ? 'プロジェクトが承認依頼中のため、新規問題報告はできません' : undefined}
placeholder={isApprovalPending ? 'プロジェクトが承認依頼中のため、コメントを追加できません' : 'コメントを入力...'}
```

- ボタンのtooltipで理由を表示
- プレースホルダーで理由を表示

---

# 13. パフォーマンス考慮事項

## 13.1 useMemoによるフィルタリング最適化

```
const filteredIssues = useMemo(() => {
  return issues.filter((issue) => {
    // フィルタリングロジック
  });
}, [issues, searchText, statusFilter, filterTaskId]);
```

**効果:**

- 依存配列が変更されない限り、再計算を避ける
- 検索テキスト入力時のパフォーマンスを改善

## 13.2 destroyOnHiddenの使用

```
<Modal destroyOnHidden>
```

**メリット:**

- モーダルが非表示になる度にコンポーネントが破棄され、状態がリセットされる
- メモリリークを防ぐ

**デメリット:**

- モーダルを開く度に再レンダリングが発生
- ただし、このモーダルは複雑だが許容範囲内

> ***Note:*** `destroyOnClose` *は旧API で非推奨。Ant Design 最新版では* `destroyOnHidden` *を使用する。*

## 13.3 テーブルのスクロール仮想化

```
scroll={{ y: 'calc(70vh - 280px)' }}
```

**効果:**

- 大量の問題がある場合でも、スクロール可能な固定高さに制限
- レンダリング対象を制限してパフォーマンスを維持

---

# 14. 備考

## 14.1 特記事項

1. **2カラムレイアウトの採用理由**
   - 左側で問題を素早く一覧表示・選択
   - 右側で詳細を確認・操作
   - 効率的な問題管理ワークフローを実現

2. **参照専用モード（viewMode）の目的**
   - 実験管理者などが問題を閲覧するのみの場合に使用
   - 編集・新規作成ボタンを非表示にして混乱を防ぐ

3. **承認依頼中の編集制限**
   - プロジェクトが `waiting_approval` ステータスの場合、編集不可
   - 承認プロセス中の変更を防ぐ
   - ツールチップで理由を明確に表示

4. **検索・フィルタリング機能の重要性**
   - 問題数が増えても素早く目的の問題を見つけられる
   - タイトルと内容の両方で検索可能
   - ステータスで絞り込み可能

5. **コメント履歴の表示**

- 過去の議論を確認できる
- 問題解決のプロセスを追跡できる
- 最大200pxの高さでスクロール可能

6. **新規問題報告フォームの項目**
   - タイトル: 100文字以内
   - 対象: プロジェクト/タスク/トランジション
   - 問題タイプ: 7種類（材料不足、機器故障など）
   - 内容: 10〜1000文字

7. **filterTaskIdの用途**
   - 特定のタスクに関連する問題のみを表示
   - 参照専用モードと組み合わせて使用
   - タスク単位での問題確認に便利

## 14.2 将来の拡張案

1. **優先度の設定**
   - 高/中/低の優先度を設定
   - 優先度で並び替え

2. **担当者の割り当て**
   - 問題の解決を特定の人に依頼
   - 担当者でフィルタリング

3. **期限の設定**
   - 問題解決の期限を設定
   - 期限切れを警告表示

4. **添付ファイル機能**
   - スクリーンショットや証拠ファイルの添付
   - 画像のプレビュー表示

5. **問題のエクスポート**
    - CSV/PDFでエクスポート
    - レポート作成に活用
6. **通知機能**
    - 新しいコメントが追加された時に通知
    - ステータスが変更された時に通知
7. **問題の統計表示**
    - 未解決/解決済の件数
    - 問題タイプ別の件数
    - グラフ表示

## 14.3 関連コンポーネント

- **IssueReportModal**: 単一問題の報告とステータス変更
- **ProgressCheckTab**: 問題管理ボタンの呼び出し元
- **InstructionConfirmationTab**: 問題管理ボタンの呼び出し元
- **Issue型**: 問題のデータ型
- **IssueStatus型**: ステータスの型定義
- **IssueTargetType型**: 対象タイプの型定義

---

# 15. 変更履歴

| バージョン | 日付 | 変更内容 | 担当者 |
|---|---|---|---|
| 1.3.0 | 2026-03-11 | 仕様書をソースコードと同期: Props設計をコールバック方式から内部API直接呼び出し方式に変更（`issues` / `onSubmitIssue` / `onSubmitComment` / `onUpdateStatus` 削除 → `projectId` / `projectName` / `targetName` / `methodId` / `taskNo` 追加）、`fetchIssues` / `issuesLoading` 追記、各イベントハンドラのAPI呼び出し処理に更新、 | Cline |

| | | | |
|---|---|---|---|
| | | `destroyOnClose` → `destroyOnHidden` 修正、`bodyStyle` → `styles.body` 修正、ボタンtooltip追記 | |
| 1.2.0 | 2026-02-27 | コメント送信後の即時表示対応: `selectedIssue` を state から `useMemo` による派生値に変更し、`issues` props 更新時に自動反映されるよう修正 | Cline |
| 1.1.0 | 2026-02-27 | 問題タイプ選択肢をハードコードからコードマスターAPI取得に変更（useCodeMaster('issue_type')） | Cline |
| 1.0.0 | 2026-01-19 | 初版作成、UI設計書として文書化 | Cline |